

LAPORAN PRAKTIKUM
STUKTUR DATA

Implementasi Binary Tree, Traversal, dan Graph Traversal (DFS & BFS)

Disusun oleh:
Thaariq Salam
2511532022

Dosen Pengampu: Dr. Wahyudi S.T.M.T
Asisten Dosen: Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan kehadiran Allah SWT, karena atas rahmat, hidayah, serta karunia-Nya, penulis dapat menyelesaikan Laporan Praktikum Struktur Data Pekan 9 dengan topik "Implementasi Binary Tree, Traversal, dan Graph Traversal (DFS & BFS)" ini dengan baik dan tepat pada waktunya.

Laporan ini disusun guna memenuhi salah satu tugas akademik pada mata kuliah Praktikum Struktur Data di Fakultas Teknologi Informasi, Universitas Andalas. Melalui laporan ini, diharapkan pembaca dapat memahami implementasi nyata dari struktur data non-linear, mekanisme penelusuran pohon biner, representasi objek graf menggunakan *Adjacency List*, serta perbandingan rute pencarian algoritma DFS dan BFS dalam bahasa pemrograman Java.

Dalam proses penyusunan laporan praktikum ini, penulis banyak mendapatkan bimbingan, arahan, serta dukungan dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan ucapan terima kasih yang sebesar-besarnya kepada:

1. Bapak Dr. Wahyudi S.T.M.T selaku Dosen Pengampu mata kuliah Struktur Data.
2. Kak Sherly Sukmadira selaku Asisten Praktikum yang telah memberikan arahan selama pelaksanaan praktikum.

Penulis menyadari bahwa laporan praktikum ini masih jauh dari kesempurnaan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi perkembangan ilmu pengetahuan, khususnya di bidang Informatika.

Padang, 5 Juni 2026

Thaariq Salam

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	2
1.3 Manfaat.....	2
BAB II PEMBAHASAN.....	3
2.1 Class Node_2511532022.java.....	3
2.1.1 Kode program.....	3
2.1.2 Langkah - langkah.....	4
2.1.3 Analisis.....	5
2.2 Class BTree_2511532022.java.....	5
2.2.1 Kode program.....	6
2.2.2 Langkah – langkah.....	7
2.2.3 Analisis.....	7
2.3 Class BtreeDriver_2511532022.java.....	7
2.3.1 Kode program.....	8
2.3.2 Langkah - langkah.....	8
2.3.3 Analisis.....	9
2.4 Class GraphTraversal_2511532022.java.....	10
2.4.1 Kode program.....	11
2.4.2 Langkah - langkah.....	12
2.4.3 Analisis.....	12
BAB III KESIMPULAN.....	14
TUGAS PRAKTIKUM STRUKTUR DATA PEKAN 9.....	15
1. Kode Program Class PetaKampus_2511532022.....	15
2. Langkah – langkah.....	19
3. Analisis Alur Eksekusi.....	20
4. Penjelasan Struktur Graph, Cara Kerja BFS, dan DFS.....	22
4.1. Struktur Graph Peta Kampus Universitas Andalas.....	22
4.2. Cara Kerja Algoritma BFS (<i>Breadth-First Search</i>).....	23
4.3. Cara Kerja Algoritma DFS (<i>Depth-First Search</i>).....	24

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan penulisan program dalam bidang Informatika menuntut pemilihan struktur data yang efisien guna memodelkan kompleksitas data di dunia nyata yang sering kali bersifat hierarkis atau saling terhubung seperti jaringan rute dan sistem berkas komputer. Berbeda dengan struktur linear (*array*, *stack*, atau *queue*) yang menyusun data berurutan, struktur data non-linear seperti *Binary Tree* (Pohon Biner) dan *Graph* (Graf) menawarkan fleksibilitas yang lebih tinggi. *Binary Tree* membatasi setiap simpul (*node*) untuk memiliki maksimal dua objek anak (kiri dan kanan), yang urutan datanya dapat ditelusuri secara mendalam memanfaatkan fungsi rekursif melalui metode *Preorder* (Root > Left > Right), *Inorder* (Left > Root > Right), ataupun *Postorder* (Left > Right > Root).

Sementara itu, *Graph* merepresentasikan jaringan hubungan antar-simpul bebas (*vertices*) yang dihubungkan oleh sisi (*edges*), di mana pemetaannya secara efisien diimplementasikan melalui senarai ketetanggaan (*Adjacency List*) menggunakan kombinasi objek Map dan List di Java. Untuk menjelajahi seluruh titik simpul graf tanpa terjebak dalam putaran tak terbatas (*cycle*), diperlukan algoritma penelusuran seperti *Depth-First Search* (DFS) yang menelusuri kedalaman cabang vertikal secara rekursif berbasis tumpukan (LIFO), serta *Breadth-First Search* (BFS) yang menjelajahi tetangga horizontal selevel secara iteratif dengan bantuan antrian (Queue bersifat FIFO). Oleh karena itu, praktikum pekan 9 ini penting dilakukan untuk memberikan pemahaman praktis dalam mengonstruksi, mengaitkan, dan melacak struktur data non-linear tersebut langsung melalui orientasi kode pemrograman Java.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut

1. Memahami karakteristik, representasi memori, dan perbedaan fundamental antara struktur data linear dan struktur data non-linear (*Tree* dan *Graph*).
2. Mampu merancang kelas objek simpul (*Node*) yang mengandalkan pointer penunjuk objek anak kiri dan anak kanan di pemrograman Java.
3. Mampu menerapkan metode kunjungan rekursif *Preorder*, *Inorder*, dan *Postorder* untuk menampilkan susunan isi data pada pohon biner secara tepat.
4. Mampu merepresentasikan objek graf tak berarah (*undirected graph*) melalui teknik senarai ketetanggaan (*Adjacency List*) memanfaatkan koleksi objek Map dan List.
5. Memahami langkah kerja, rute pelacakan, serta mengimplementasikan algoritma DFS rekursif dan BFS iteratif berbasis antrean pada graf.

1.3 Manfaat

Manfaat yang didapat dari praktikum ini adalah sebagai berikut.

1. Mengasah logika analitis dalam memecahkan masalah pemetaan data non-linear serta memantapkan keahlian dalam memanfaatkan fungsi rekursif objek di Java.
2. Memahami dasar algoritma krusial yang digunakan pada sistem mesin pencari (*web crawler*), penentuan rute terpendek peta digital, hingga struktur pengindeksan dalam basis data komputer,
3. Menjadi pondasi pengetahuan algoritma tingkat lanjut yang dapat dimanfaatkan untuk pengembangan perangkat lunak kompleks, kecerdasan buatan (*AI graph search*), maupun pengerjaan tugas akhir di program studi Informatika ke depannya.

BAB II

PEMBAHASAN

2.1 Class Node_2511532022.java

Class Node_2511532022 merupakan cetak biru (*blueprint*) dasar untuk merepresentasikan sebuah simpul tunggal di dalam struktur *Binary Tree*. Setiap objek dari class ini menyimpan satu buah data bertipe integer (*data_2022*) serta dua buah variabel referensi objek (*left_2022* dan *right_2022*) yang berfungsi sebagai *pointer* menuju anak kiri dan anak kanan dari simpul tersebut. Class ini juga memuat fungsi pencetakan bentuk pohon dan metode kunjungan (*traversal*) dasar secara rekursif.

2.1.1 Kode program

```
1 package pekan9_2511532022;
2
3 public class Node_2511532022 {
4     int data_2022;
5     Node_2511532022 left_2022;
6     Node_2511532022 right_2022;
7
8     public Node_2511532022(int data) {
9         this.data_2022 = data;
10        left_2022 = null;
11        right_2022 = null;
12    }
13    // SETTER
14    public void setLeft(Node_2511532022 node) {
15        if (left_2022 == null)
16            left_2022 = node;
17    }
18    public void setRight(Node_2511532022 node) {
19        if (right_2022 == null)
20            right_2022 = node;
21    }
22    public void setData(int data) {
23        this.data_2022 = data;
24    }
25
26    // GETTER
27    public Node_2511532022 getLeft() {
28        return left_2022;
29    }
30    public Node_2511532022 getRight() {
31        return right_2022;
32    }
33    public int getData() {
34        return data_2022;
35    }
36}
```

```

37 // Traversal Methods
38 void printPreorder(Node_2511532022 node) {
39     if (node == null)
40         return;
41     System.out.print(node.data_2022 + " ");
42     printPreorder(node.left_2022);
43     printPreorder(node.right_2022);
44 }
45 void printPostorder(Node_2511532022 node) {
46     if (node == null)
47         return;
48     printPostorder(node.left_2022);
49     printPostorder(node.right_2022);
50     System.out.print(node.data_2022 + " ");
51 }
52 void printInorder(Node_2511532022 node) {
53     if (node == null)
54         return;
55     printInorder(node.left_2022);
56     System.out.print(node.data_2022 + " ");
57     printInorder(node.right_2022);
58 }
59
60 public String print() {
61     return this.print("", true, "");
62 }
63 public String print(String prefix, boolean isTail, String sb) {
64     if (right_2022 != null) {
65         right_2022.print(prefix + (isTail ? "| " : " "), false, sb);
66     }
67     System.out.println(prefix + (isTail ? "\\-- " : "/--") + data_2022);
68
69     if (left_2022 != null) {
70         left_2022.print(prefix + (isTail ? " " : "| "), true, sb);
71     }
72     return sb;
73 }
74 }
75 }

```

2.1.2 Langkah - langkah

1. Mendefinisikan atribut data_2022, left_2022, dan right_2022.
2. Membuat konstruktor untuk menginisialisasi nilai data baru serta memberikan nilai awal null pada simpul cabang (kiri dan kanan).
3. Membuat metode *getter* dan *setter* bersyarat (if (left_2022 == null)) untuk memastikan simpul anak tidak tertimpa secara tidak sengaja ketika sudah terisi.
4. Merancang algoritma kunjungan rekursif *Preorder*, *Inorder*, dan *Postorder* dengan basis berhentinya fungsi (*base case*) berupa if (node == null) return;.
5. Mengimplementasikan fungsi pencetakan berbasis string format teks guna memvisualisasikan orientasi percabangan kanan-kiri ke layar secara hirarkis.

2.1.3 Analisis

Pembatasan percabangan di dalam konstruktor dan fungsi *setter* menjamin struktur pohon yang terbentuk adalah *Binary Tree* murni (maksimal berderajat 2). Proses penelusuran data memanfaatkan *call stack* internal JVM secara rekursif. Karakteristik utama dari metode *traversal* di dalam objek simpul ini ditentukan oleh peletakan baris perintah `System.out.print(node.data_2022 + " ");`:

- Preorder: Data dicetak di awal sebelum fungsi memanggil anak kiri dan anak kanan.
- Inorder: Data dicetak di tengah, tepat setelah kunjungan anak kiri selesai dan sebelum bergerak ke anak kanan.
- Postorder: Data dicetak di bagian paling akhir setelah kedua cabang selesai dieksplorasi penuh.

2.2 Class BTree_2511532022.java

Class BTree_2511532022 bertindak sebagai pembungkus (*wrapper class*) utama yang mengelola struktur makro dari pohon biner tersebut. Di dalamnya terdapat variabel penunjuk simpul utama (`root_2022`) serta simpul navigasi aktif (`currentNode_2022`). *Class* ini bertugas mendelegasikan pemanggilan metode eksternal seperti menghitung jumlah simpul secara dinamis, melakukan pencarian nilai data (`search`), memeriksa kekosongan pohon, serta membungkus antarmuka cetak transversal.

2.2.1 Kode program

```
1 package pekan9_2511532022;
2
3 public class BTree_2511532022 {
4     private Node_2511532022 root_2022;
5     private Node_2511532022 currentNode_2022;
6     public BTree_2511532022() {
7         root_2022 = null;
8     }
9     public boolean search(int data) {
10         return search(root_2022, data);
11     }
12     private boolean search(Node_2511532022 node, int data) {
13
14         if (node.getData() == data)
15             return true;
16         if (node.getLeft() != null)
17             if (search(node.getLeft(), data))
18                 return true;
19         if (node.getRight() != null)
20             if (search(node.getRight(), data))
21                 return true;
22         return false;
23     }
24     public void printInorder() {
25         root_2022.printInorder(root_2022);
26     }
27     public void printPreOrder() {
28         root_2022.printPreorder(root_2022);
29     }
30     public void printPostOrder() {
31         root_2022.printPostorder(root_2022);
32     }
33
34     public Node_2511532022 getRoot() {
35         return root_2022;
36     }
37
38     public boolean isEmpty() {
39         return root_2022 == null;
40     }
41
42     public int countNodes() {
43         return countNodes(root_2022);
44     }
45
46     private int countNodes(Node_2511532022 node) {
47         int count = 1;
48         if (node == null) {
49             return 0;
50         } else {
51             count += countNodes (node.getLeft());
52             count += countNodes (node.getRight());
53             return count;
54         }
55     }
56
57     public void print() {
58         root_2022.print();
59     }
60     public Node_2511532022 getCurrent() {
61         return currentNode_2022;
62     }
63
64     public void setCurrent(Node_2511532022 node) {
65         this.currentNode_2022 = node;
66     }
67
68     public void setRoot(Node_2511532022 root) {
69         this.root_2022 = root;
70     }
71 }
```

2.2.2 Langkah – langkah

1. Mendefinisikan enkapsulasi data private Node_2511532022 root_2022 dan currentNode_2022.
2. Menulis fungsi pencarian search(int data) publik yang memicu metode pencarian privat bermetode rekursif *overloading* dengan melemparkan parameter acuan awal dari root_2022.
3. Merancang fungsi penghitung simpul countNodes yang akan mengembalikan nilai 0 jika simpul kosong, atau menjumlahkan secara akumulatif komponen $1 + \text{countNodes}(\text{left}) + \text{countNodes}(\text{right})$.
4. Mengintegrasikan fungsi publik (printPreOrder, printInorder, printPostOrder) agar langsung memanggil metode milik kelas simpul yang terhubung.

2.2.3 Analisis

Class ini mengaplikasikan konsep pemisahan logika yang baik (*Separation of Concerns*). Pengguna luar tidak perlu melakukan penelusuran manual terhadap referensi objek simpul; mereka cukup memanggil antarmuka dari objek BTree_2511532022. Fungsi pencarian search di kelas ini melakukan penelusuran secara mendalam di seluruh wilayah pohon biner tidak terurut hingga menemukan kecocokan nilai, menghasilkan kompleksitas waktu terburuk (*Worst-Case Time Complexity*) senilai $O(n)$ di mana n adalah jumlah simpul.

2.3 Class BtreeDriver_2511532022.java

Class BtreeDriver_2511532022 adalah kelas eksekusi utama (*Main Class*) yang menguji fungsionalitas dari pohon biner. Kelas ini menginstansiasi objek pohon baru, membangun interkoneksi fisik antar-simpul berangka 1 sampai 9 secara manual dengan menetapkan hubungan anak-kiri (*left*) dan anak-kanan (*right*), kemudian

memverifikasi hasil kalkulasi simpul dan pencetakan ketiga urutan kunjungan data.

2.3.1 Kode program

```
1 package pekan9_2511532022;
2
3 public class BtreeDriver_2511532022 {
4     public static void main (String[] args) {
5         // Membuat Pohon
6         BTree_2511532022 tree = new BTree_2511532022();
7         System.out.print("Jumlah Simpul awal pohon: ");
8         System.out.println(tree.countNodes());
9
10        //Menambahkan simpul data 1
11        Node_2511532022 root = new Node_2511532022 (1);
12        // Menjadikan simpul 1 sebagai root
13        tree.setRoot (root);
14        System.out.println("Jumlah simpul jika hanya ada root");
15        System.out.println(tree.countNodes());
16        Node_2511532022 node2 = new Node_2511532022 (2);
17        Node_2511532022 node3 = new Node_2511532022 (3);
18        Node_2511532022 node4 = new Node_2511532022 (4);
19        Node_2511532022 node5 = new Node_2511532022 (5);
20        Node_2511532022 node6 = new Node_2511532022 (6);
21        Node_2511532022 node7 = new Node_2511532022 (7);
22        Node_2511532022 node8 = new Node_2511532022 (8);
23        Node_2511532022 node9 = new Node_2511532022 (9);
24        root.setLeft(node2);
25        node2.setLeft(node4);
26        node2.setRight(node5);
27        node4.setRight(node8);
28        root.setRight(node3);
29        node3.setLeft(node6);
30        node3.setRight(node7);
31        node6.setLeft(node9);
32
33        //Set root
34        tree.setCurrent(tree.getRoot());
35        System.out.println("menampilkan simpul terakhir: ");
36        System.out.println(tree.getCurrent().getData());
37        System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
38        System.out.println(tree.countNodes());
39        System.out.println("InOrder: ");
40        tree.printInorder();
41        System.out.println("\nPreorder: ");
42        tree.printPreOrder();
43        System.out.println("\nPostorder: ");
44        tree.printPostOrder();
45        System.out.println("\nMenampilkan simpul dalam bentuk pohon");
46        tree.print();
47    }
48 }
```

2.3.2 Langkah - langkah

1. Membuat instansiasi pohon kosong BTree_2511532022 tree.

2. Membuat instansiasi objek simpul utama bernilai 1 dan menetapkan sebagai Root.
3. Menginstansiasi objek simpul node2 hingga node9 dengan muatan data masing-masing angka.
4. Mengaitkan relasi pointer hierarki struktur pohon melalui perintah pencabangan:
5. Cabang kiri akar utama: $1 > 2 > 4 > 8$ (anak kanan dari 4) dan $2 > 5$.
6. Cabang kanan akar utama: $1 > 3 > 6 > 9$ (anak kiri dari 6) dan $3 > 7$.
7. Mengeksekusi pencetakan total node aktif dan pemanggilan ketiga fungsi penelusuran (*traversal*) data pohon.

2.3.3 Analisis

Berdasarkan konfigurasi manual pada fungsi utama, diperoleh visualisasi pemetaan pohon biner asli sebagai berikut:

- Simpul akar utama: `1`
- Sisi kiri: Sub-pohon yang memiliki struktur internal di mana simpul `2` menjadi induk bagi daun `4` dan `5`. Simpul `4` memiliki anak kanan yaitu `8`.
- Sisi kanan: Sub-pohon yang memiliki struktur internal di mana simpul `3` menjadi induk bagi daun `6` dan `7`. Simpul `6` memiliki anak kiri yaitu `9`.

Kalkulasi jumlah simpul terbukti dinamis: bernilai `0` di awal, menjadi `1` ketika root didaftarkan, dan bernilai `9` saat semua simpul selesai dikaitkan. Hasil log teks konsol menunjukkan urutan traversal yang tepat:

- Inorder (Kiri-Root-Kanan) : [8, 4, 2, 5, 1, 9, 6, 3, 7]
- Preorder (Root-Kiri-Kanan) : [1, 2, 4, 8, 5, 3, 6, 9, 7]
- Postorder (Kiri-Kanan-Root) : [8, 4, 5, 2, 9, 6, 7, 3, 1]

2.4.1 Kode program

```
1 package pekan9_2511532022;
2
3 import java.util.*;
4
5 public class GraphTraversal_2511532022 {
6     private Map<String, List<String>> graph_2022 = new HashMap<>();
7
8     // Menambahkan edge (graf tak berarah)
9     public void addEdge (String node1, String node2) {
10         graph_2022.putIfAbsent(node1, new ArrayList<>());
11         graph_2022.putIfAbsent(node2, new ArrayList<>());
12         graph_2022.get (node1).add(node2);
13         graph_2022.get(node2).add(node1);
14     }
15     // Menampilkan graf awal
16     public void printGraph() {
17         System.out.println("Graf Awal (Adjacency List): ");
18         for (String node : graph_2022.keySet() ) {
19             System.out.print(node + " -> ");
20             List<String> neighbors = graph_2022.get(node);
21             System.out.println(String.join(", ", neighbors));
22         }
23         System.out.println();
24     }
25
26     // DFS rekursif
27     public void dfs (String start) {
28         Set<String> visited = new HashSet<>();
29         System.out.println("Penelusuran DFS: ");
30         dfsHelper(start, visited);
31         System.out.println();
32     }
33     private void dfsHelper(String current, Set<String> visited) {
34         if (visited.contains(current)) return;
35         visited.add(current);
36         System.out.print(current + " ");
37         for (String neighbor : graph_2022.getOrDefault(current, new ArrayList<>())) {
38             dfsHelper(neighbor, visited);
39         }
40     }
41
42     // BFS iteratif
43     public void bfs(String start) {
44         Set<String> visited = new HashSet<>();
45         Queue<String> queue = new LinkedList<>();
46         queue.add(start);
47         visited.add(start);
48         System.out.println("Penelusuran BFS:");
49         while (!queue.isEmpty()) {
50             String current = queue.poll();
51             System.out.print(current + " ");
52             for (String neighbor : graph_2022.getOrDefault(current, new ArrayList<>())) {
53                 if (!visited.contains(neighbor)) {
54                     queue.add(neighbor);
55                     visited.add(neighbor);
56                 }
57             }
58         }
59         System.out.println();
60     }
61
62     // Main
63     public static void main (String[] args) {
64         GraphTraversal_2511532022 graph = new GraphTraversal_2511532022();
65
66         // Contoh graf: A-B, A-C, B-D, B-E
67         graph.addEdge("A", "B");
68         graph.addEdge("A", "C");
69         graph.addEdge("B", "D");
70         graph.addEdge("B", "E");
71         // Cetak graff awal
72         System.out.println("Graf Awal adalah: ");
73         graph.printGraph();
74         // Lakukan penelusuran
75         graph.dfs("A");
76         graph.bfs("A");
77     }
78 }
```

2.4.2 Langkah - langkah

1. Membuat penampung graf berbasis HashMap bertipe `Map<String, List<String>>`.
2. Membuat metode `addEdge` yang secara otomatis memesan ruang larik baru (`putIfAbsent`) untuk kedua simpul yang saling terhubung, lalu memasukkan nama simpul tujuan ke daftar tetangga masing-masing simpul guna membentuk relasi dua arah balik (*undirected edge*).
3. Merancang metode pencarian DFS dengan membuat koleksi data unik (`Set<String> visited`) untuk mencegah bahaya perulangan tak terbatas (*infinite loop/cycle*), kemudian menelusuri simpul tetangga terdalam menggunakan fungsi bantu `dfsHelper` secara rekursif.
4. Merancang metode pencarian BFS secara iteratif dengan memasukkan elemen awal ke dalam antrean objek `Queue`, melakukan pengosongan antrean lewat operasi `poll()`, lalu mendaftarkan tetangga selevel yang belum pernah dikunjungi ke ujung antrean.
5. Membangun struktur hubungan antar titik simpul di dalam fungsi utama (A-B, A-C, B-D, B-E) dan mengeksekusi penelusuran yang dimulai dari titik awal "A".

2.4.3 Analisis

Metode pembentukan *Adjacency List* menggunakan kamus (`Map`) sangat efisien dalam menghemat alokasi penggunaan memori komputer dibandingkan dengan penggunaan matriks ketetanggaan (*Adjacency Matrix*), terutama jika graf bersifat jarang (*sparse graph*). Analisis penelusuran rute pelacakan dari titik awal "A" menghasilkan keluaran logika berikut:

- DFS (A B D E C): Bergerak maju secara linear menyusuri kedalaman cabang terjauh. Dari simpul A beralih ke B, lalu masuk ke anak pertama B yaitu D. Karena simpul D tidak memiliki tetangga baru lagi, fungsi melakukan langkah

mundur (*backtracking*) ke simpul B, meneruskan jalur ke E, dan terakhir kembali ke awal untuk melompat ke simpul C.

- BFS (A B C D E): Bergerak melebar secara horizontal lapis demi lapis (per level tetangga langsung). Mula-mula simpul A dikunjungi, lalu mengeksplorasi seluruh tetangga dekat berlevel sama milik A (yaitu B dan C). Setelah level tersebut habis, antrian akan memproses anak tetangga berikutnya dari simpul B yang paling depan, yaitu simpul D dan E.

```
<terminated> GraphTraversal_2511532022
```

```
Graf Awal adalah:
```

```
Graf Awal (Adjacency List):
```

```
A -> B, C
```

```
B -> A, D, E
```

```
C -> A
```

```
D -> B
```

```
E -> B
```

```
Penelusuran DFS:
```

```
A B D E C
```

```
Penelusuran BFS:
```

```
A B C D E
```


BAB III

KESIMPULAN

1. Struktur data non-linear (*Binary Tree* dan *Graph*) sangat ideal digunakan untuk merepresentasikan masalah pemetaan data yang memiliki hubungan hierarkis ataupun hubungan jaringan yang kompleks secara dinamis.
2. Penggunaan fungsi rekursif sangat mempermudah penulisan instruksi kode *traversal* pohon (*Preorder*, *Inorder*, *Postorder*) karena karakteristik struktur dari setiap sub-pohon di dalam *Binary Tree* selalu identik dengan induk utamanya.
3. Struktur data koleksi Map dan List di Java sangat fleksibel dan efisien dalam membentuk representasi senarai ketetanggaan (*Adjacency List*) untuk memodelkan struktur graf tak berarah tanpa membuang-buang memori kosong.
4. Perbedaan mendasar antara kedua metode pelacakan graf terletak pada prinsip pengelolaan struktur data pembantunya: DFS menyusuri kedalaman jalur vertikal dengan memanfaatkan tumpukan rekursif sistem (*Stack* bersifat LIFO), sedangkan BFS mengutamakan perluasan area horizontal per lapis dengan memanfaatkan bantuan antrean objek (*Queue* bersifat FIFO).

TUGAS PRAKTIKUM STRUKTUR DATA PEKAN 9

Tugas BFS dan DFS

1. Kode Program Class PetaKampus_2511532022

```
1 package pekan9_2511532022;
2
3 import javax.swing.*;
4 import javax.swing.border.TitledBorder;
5 import java.awt.*;
6 import java.util.*;
7 import java.util.List;
8
9 public class PetaKampus_2511532022 extends JFrame {
10     private Map<String, List<String>> graph_2022 = new HashMap<>();
11     private Map<String, Point> positions_2022 = new HashMap<>();
12     private List<String> path_2022 = new ArrayList<>();
13     private List<String> visitedOrder_2022 = new ArrayList<>();
14
15     private JComboBox<String> startCombo_2022;
16     private JComboBox<String> goalCombo_2022;
17     private JTextArea resultArea_2022;
18     private JPanel canvas_2022;
19
20     public PetaKampus_2511532022() {
21         setTitle("PENCAIRAN JALUR MENGGUNAKAN BFS DAN DFS - 2511532022");
22         setSize(900, 700);
23         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
24         setLocationRelativeTo(null);
25
26         initGraphData_2022();
27         initComponents_2022();
28     }
29
30     private void initGraphData_2022() {
31         // Pemetaan posisi node koordinat (X, Y) disesuaikan agar menyebar membentuk peta
32         positions_2022.put("Gerbang", new Point(60, 240));
33         positions_2022.put("Rektorat", new Point(220, 140));
34         positions_2022.put("Auditorium", new Point(220, 340));
35         positions_2022.put("Masjid", new Point(380, 240));
36         positions_2022.put("Perpus", new Point(380, 80));
37         positions_2022.put("FTI", new Point(540, 140));
38         positions_2022.put("Dekanat", new Point(540, 340));
39         positions_2022.put("GedungA", new Point(700, 80));
40         positions_2022.put("GedungB", new Point(700, 240));
41         positions_2022.put("LaborAl", new Point(800, 340));
42     }
```

```

43-   for (String node : positions_2022.keySet()) {
44       graph_2022.put(node, new ArrayList<>());
45   }
46
47   // Relasi 16 Sisi (Edge) Tak Berarah
48   addEdge_2022("Gerbang", "Rektorat");
49   addEdge_2022("Gerbang", "Auditorium");
50   addEdge_2022("Rektorat", "Perpus");
51   addEdge_2022("Rektorat", "Masjid");
52   addEdge_2022("Auditorium", "Masjid");
53   addEdge_2022("Auditorium", "Dekanat");
54   addEdge_2022("Perpus", "FTI");
55   addEdge_2022("Perpus", "GedungA");
56   addEdge_2022("Masjid", "FTI");
57   addEdge_2022("Masjid", "Dekanat");
58   addEdge_2022("FTI", "GedungA");
59   addEdge_2022("FTI", "GedungB");
60   addEdge_2022("Dekanat", "GedungB");
61   addEdge_2022("Dekanat", "LaborAL");
62   addEdge_2022("GedungA", "GedungB");
63   addEdge_2022("GedungB", "LaborAL");
64
65   // Pengurutan alfabetis tetangga simpul
66-   for (String node : graph_2022.keySet()) {
67       Collections.sort(graph_2022.get(node));
68   }
69   }
70
71-   private void addEdge_2022(String u, String v) {
72       graph_2022.get(u).add(v);
73       graph_2022.get(v).add(u);
74   }
75
76-   private void initComponents_2022() {
77       setLayout(new BorderLayout(10, 10));
78
79       // 1. PANEL ATAS (Header Judul & Input Kontrol)
80       JPanel topPanel_2022 = new JPanel(new GridBagLayout());
81       topPanel_2022.setBackground(new Color(240, 244, 248));
82       GridBagConstraints gbc_2022 = new GridBagConstraints();
83       gbc_2022.insets = new Insets(5, 10, 5, 10);
84
85       gbc_2022.gridx = 0;
86       gbc_2022.gridy = 0;
87       gbc_2022.gridwidth = GridBagConstraints.REMAINDER;
88
89       // Label Judul Utama Besar
90       JLabel titleLabel_2022 = new JLabel("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS");
91       titleLabel_2022.setFont(new Font("Arial", Font.BOLD, 18));
92       titleLabel_2022.setForeground(new Color(44, 62, 80));
93       topPanel_2022.add(titleLabel_2022, gbc_2022);
94
95       // Baris Input Kontrol
96       gbc_2022.gridwidth = 1;
97       gbc_2022.gridy = 1;
98
99       String[] nodes_2022 = positions_2022.keySet().toArray(new String[0]);
100       Arrays.sort(nodes_2022);
101       startCombo_2022 = new JComboBox<>(nodes_2022);
102       goalCombo_2022 = new JComboBox<>(nodes_2022);
103
104       JButton btnBFS_2022 = new JButton("BFS");
105       JButton btnDFS_2022 = new JButton("DFS");
106       JButton btnReset_2022 = new JButton("RESET");
107
108       gbc_2022.gridx = 0; topPanel_2022.add(new JLabel("Lokasi Awal:"), gbc_2022);
109       gbc_2022.gridx = 1; topPanel_2022.add(startCombo_2022, gbc_2022);
110       gbc_2022.gridx = 2; topPanel_2022.add(new JLabel("Lokasi Tujuan:"), gbc_2022);
111       gbc_2022.gridx = 3; topPanel_2022.add(goalCombo_2022, gbc_2022);
112       gbc_2022.gridx = 4; topPanel_2022.add(btnBFS_2022, gbc_2022);
113       gbc_2022.gridx = 5; topPanel_2022.add(btnDFS_2022, gbc_2022);
114       gbc_2022.gridx = 6; topPanel_2022.add(btnReset_2022, gbc_2022);
115
116       add(topPanel_2022, BorderLayout.NORTH);
117
118       // 2. PANEL TENGAH (Visualisasi Graph dengan TitledBorder)
119       canvas_2022 = new JPanel() {
120           @Override
121           protected void paintComponent(Graphics g) {
122               super.paintComponent(g);
123               Graphics2D g2 = (Graphics2D) g;
124               g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
125           }
126       };

```

```

125 // Menggambar Sisi (Edge)
126 g2.setColor(Color.LIGHT_GRAY);
127 g2.setStroke(new BasicStroke(2));
128 Set<String> drawn_2022 = new HashSet<>();
129 for (String node : graph_2022.keySet()) {
130     Point p1 = positions_2022.get(node);
131     for (String neighbor : graph_2022.get(node)) {
132         if (!drawn_2022.contains(neighbor + "." + node)) {
133             Point p2 = positions_2022.get(neighbor);
134             if (isEdgeInPath_2022(node, neighbor)) {
135                 g2.setColor(new Color(231, 76, 60)); // Merah tebal untuk jalur hasil rute
136                 g2.setStroke(new BasicStroke(4));
137                 g2.drawLine(p1.x, p1.y, p2.x, p2.y);
138                 g2.setColor(Color.LIGHT_GRAY);
139                 g2.setStroke(new BasicStroke(2));
140             } else {
141                 g2.drawLine(p1.x, p1.y, p2.x, p2.y);
142             }
143             drawn_2022.add(node + "." + neighbor);
144         }
145     }
146 }
147
148 // Menggambar Simpul (Node) berbentuk lingkaran
149 for (String node : positions_2022.keySet()) {
150     Point p = positions_2022.get(node);
151     if (path_2022.contains(node)) {
152         g2.setColor(new Color(46, 204, 113)); // Hijau: Node bagian jalur akhir
153     } else if (visitedOrder_2022.contains(node)) {
154         g2.setColor(new Color(241, 196, 15)); // Kuning: Sempat dieksplorasi
155     } else {
156         g2.setColor(new Color(52, 73, 94)); // Biru gelap default
157     }
158
159     g2.fillOval(p.x - 20, p.y - 20, 40, 40);
160     g2.setColor(Color.BLACK);
161     g2.drawOval(p.x - 20, p.y - 20, 40, 40);
162
163     // Label Teks Nama Node Lokasi
164     g2.setFont(new Font("Arial", Font.BOLD, 12));
165     g2.drawString(node, p.x - 22, p.y - 25);
166 }

```

```

167 }
168 };
169 canvas_2022.setBackground(Color.WHITE);
170
171 // Pembuatan Border Kotak bertuliskan "VISUALISASI GRAPH" sesuai petuniuk tugas
172 TitledBorder border_2022 = BorderFactory.createTitledBorder(
173     BorderFactory.createLineBorder(Color.GRAY, 2), "VISUALISASI GRAPH");
174 border_2022.setTitleFont(new Font("Arial", Font.BOLD, 14));
175 canvas_2022.setBorder(border_2022);
176
177 add(canvas_2022, BorderLayout.CENTER);
178
179 // 3. PANEL BAWAH (Hasil Pencarian Jalur)
180 resultArea_2022 = new JTextArea(5, 50);
181 resultArea_2022.setEditable(false);
182 resultArea_2022.setFont(new Font("Monospaced", Font.BOLD, 13));
183 resultArea_2022.setBackground(new Color(245, 245, 245));
184
185 TitledBorder resultBorder_2022 = BorderFactory.createTitledBorder(
186     BorderFactory.createLineBorder(Color.GRAY, 1), "HASIL PENCARIAN JALUR");
187 resultArea_2022.setBorder(resultBorder_2022);
188
189 add(new JScrollPane(resultArea_2022), BorderLayout.SOUTH);
190
191 // Hubungkan fungsi klik tombol ke fungsi utama
192 btnBFS_2022.addActionListener(e -> BFS_2022());
193 btnDFS_2022.addActionListener(e -> DFS_2022());
194 btnReset_2022.addActionListener(e -> resetGraph_2022());
195 }
196
197 private boolean isEdgeInPath_2022(String u, String v) {
198     for (int i = 0; i < path_2022.size() - 1; i++) {
199         if ((path_2022.get(i).equals(u) && path_2022.get(i+1).equals(v)) ||
200             (path_2022.get(i).equals(v) && path_2022.get(i+1).equals(u))) {
201             return true;
202         }
203     }
204     return false;
205 }
206
207 public void BFS_2022() {

```

```

208     resetData_2022();
209     String start = (String) startCombo_2022.getSelectedItemAt();
210     String goal = (String) goalCombo_2022.getSelectedItemAt();
211
212     Queue<String> queue_2022 = new LinkedList<>();
213     Map<String, String> parent_2022 = new HashMap<>();
214     Set<String> visited_2022 = new HashSet<>();
215
216     queue_2022.add(start);
217     visited_2022.add(start);
218     boolean found_2022 = false;
219
220     while (!queue_2022.isEmpty()) {
221         String current = queue_2022.poll();
222         visitedOrder_2022.add(current);
223
224         if (current.equals(goal)) {
225             found_2022 = true;
226             break;
227         }
228
229         for (String neighbor : graph_2022.get(current)) {
230             if (!visited_2022.contains(neighbor)) {
231                 visited_2022.add(neighbor);
232                 parent_2022.put(neighbor, current);
233                 queue_2022.add(neighbor);
234             }
235         }
236     }
237     if (found_2022) reconstructPath_2022(start, goal, parent_2022);
238     displayPath_2022("Breadth First Search (BFS)", found_2022);
239 }
240
241 public void DFS_2022() {
242     resetData_2022();
243     String start = (String) startCombo_2022.getSelectedItemAt();
244     String goal = (String) goalCombo_2022.getSelectedItemAt();
245
246     Map<String, String> parent_2022 = new HashMap<>();
247     Set<String> visited_2022 = new HashSet<>();
248

```

```

249     dfsHelper_2022(start, visited_2022, parent_2022);
250
251     boolean found_2022 = visitedOrder_2022.contains(goal);
252     if (found_2022) reconstructPath_2022(start, goal, parent_2022);
253     displayPath_2022("Depth First Search (DFS)", found_2022);
254 }
255
256 private void dfsHelper_2022(String current, Set<String> visited, Map<String, String> parent) {
257     visited.add(current);
258     visitedOrder_2022.add(current);
259
260     for (String neighbor : graph_2022.get(current)) {
261         if (!visited.contains(neighbor)) {
262             parent.put(neighbor, current);
263             dfsHelper_2022(neighbor, visited, parent);
264         }
265     }
266 }
267
268 private void reconstructPath_2022(String start, String goal, Map<String, String> parent) {
269     String curr = goal;
270     while (curr != null) {
271         path_2022.add(0, curr);
272         curr = parent.get(curr);
273     }
274 }
275
276 public void displayPath_2022(String algorithm, boolean found) {
277     StringBuilder sb_2022 = new StringBuilder();
278     sb_2022.append(" Algoritma Pencarian : ").append(algorithm).append("\n");
279     sb_2022.append(" Urutan Node Dikunjungi : ").append(String.join(" -> ", visitedOrder_2022)).append("\n");
280     sb_2022.append(" Jumlah Node Dieksplorasi : ").append(visitedOrder_2022.size()).append(" Node\n");
281     if (found) {
282         sb_2022.append(" Jalur Solusi Utama : ").append(String.join(" -> ", path_2022));
283     } else {
284         sb_2022.append(" Jalur Solusi Utama : Rute tidak ditemukan.");
285     }
286     resultArea_2022.setText(sb_2022.toString());
287     canvas_2022.repaint();
288 }
289

```

```

290 public void resetGraph_2022() {
291     resetData_2022();
292     resultArea_2022.setText(" [Info] State graf berhasil dibersihkan ke kondisi semula.");
293     canvas_2022.repaint();
294 }
295
296 private void resetData_2022() {
297     path_2022.clear();
298     visitedOrder_2022.clear();
299 }
300
301 public static void main(String[] args) {
302     SwingUtilities.invokeLater(() -> {
303         new PetaKampus_2511532022().setVisible(true);
304     });
305 }
306 }

```

2. Langkah – langkah

1. Menyiapkan kerangka dasar berupa kelas turunan dari JFrame dan mendeklarasikan struktur objek Map untuk senarai ketetanggaan graf, pemetaan posisi piksel, urutan kunjungan, dan jalur solusi.
2. Memetakan 10 koordinat lokasi kampus Universitas Andalas secara spasial dan meregistrasikan 16 buah relasi sisi jalan tak berarah melalui pemanggilan rutin fungsi addEdge_2022.
3. Memanfaatkan instruksi perulangan Collections.sort() segera setelah semua data sisi terdaftar untuk mengondisikan agar seluruh larik daftar simpul tetangga tersusun rapi mengikuti urutan alfabetis (A-Z).
4. Mengimplementasikan logika pemecahan masalah pencarian BFS_2022 berbasis antrian (Queue diwakili struktur LinkedList) untuk menelusuri simpul selevel lapis demi lapis horizontal.
5. Mengimplementasikan fungsi rekursif mendalam dfsHelper_2022 untuk melacak rute vertikal graf secara utuh hingga memproses seluruh simpul yang saling terhubung.
6. Mengaitkan komponen kontrol interaktif (JButton) ke fungsi logika masing-masing, serta menuliskan instruksi pemulihan memori pada modul resetGraph_2022.

3. Analisis Alur Eksekusi

Penambahan penyortiran urutan alfabetis terbukti menjadi faktor penentu keakuratan jalannya eksekusi program. Ketika tombol **BFS** dijalankan dengan titik awal "Gerbang" menuju target "LaborAl", algoritma akan memasukkan "Gerbang" ke dalam antrian. Karena tetangga "Gerbang" telah diurutkan menjadi ["Auditorium", "Rektorat"], maka "Auditorium" akan diproses terlebih dahulu sebelum beralih ke "Rektorat". Sifat BFS yang meluas lapis demi lapis menjamin jalur solusi utama (Gerbang -> Auditorium -> Dekanat -> LaborAl) yang ditemukan merupakan jalur terpendek berdasarkan jumlah lompatan sisi (*shortest path*). Log teks secara dinamis mencetak urutan node tereksplorasi secara bertahap pada area resultArea_2022.

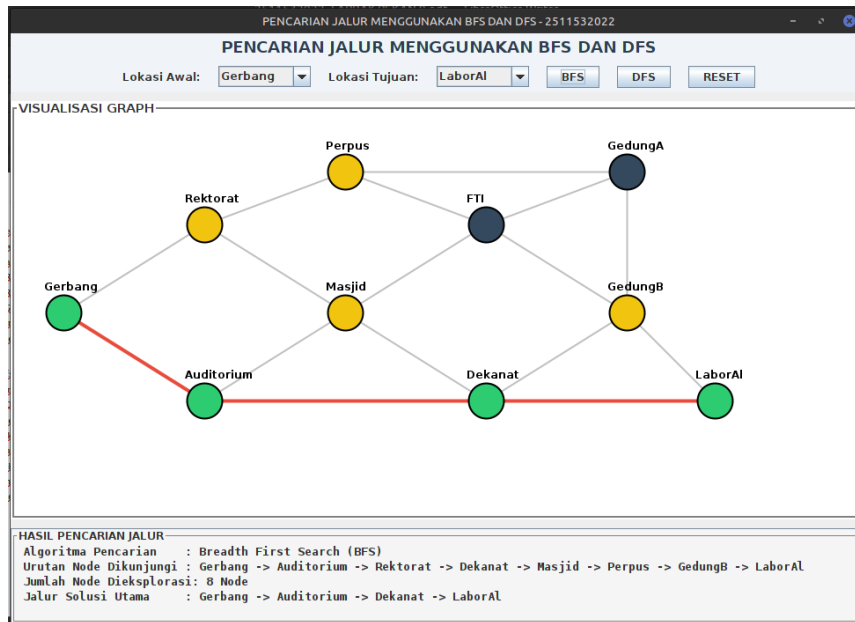
Sebaliknya, saat tombol **DFS** ditekan, program akan memicu fungsi dfsHelper_2022 secara rekursif. Karena sifat penjelajahannya yang mendalam, dari simpul "Gerbang" ia langsung terjun menuju "Auditorium", dilanjutkan ke "Dekanat", kemudian menyusuri cabang "GedungB". Berhubung fungsi rekursif mengeksplorasi seluruh komponen graf yang terhubung sebelum menyelesaikan siklus tumpukannya (*call stack*), seluruh **10 Node** akan tercatat telah dikunjungi di dalam log urutan. Setelah itu, fungsi reconstructPath_2022 akan menelusuri kembali rekam jejak peta relasi induk (*parent map*) secara mundur dari titik target "LaborAl" untuk menarik garis lurus menuju titik awal, menghasilkan warna hijau tebal yang menunjukkan rute final yang berhasil terbentuk.

Percobaan :

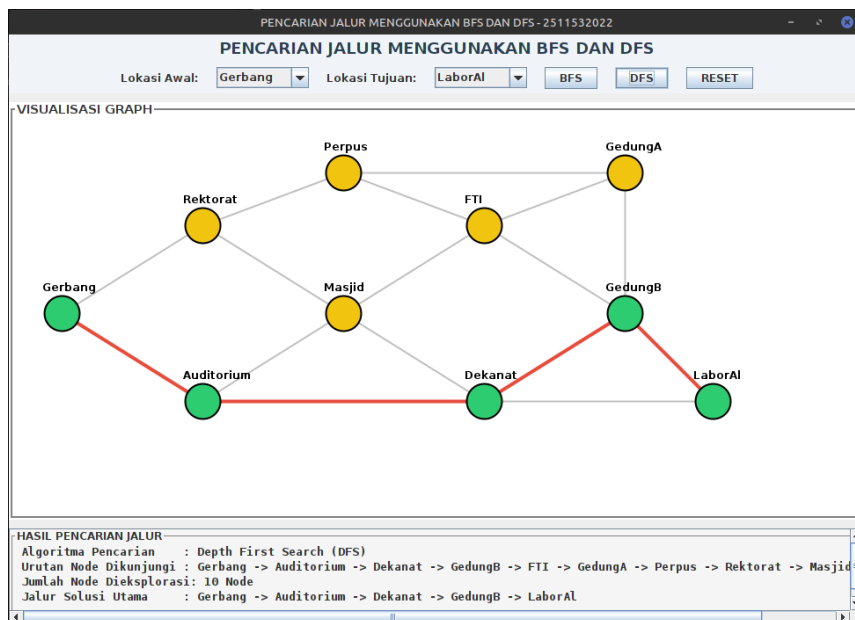
Lokasi awal : Gerbang

Lokasi Tujuan : Labor AI

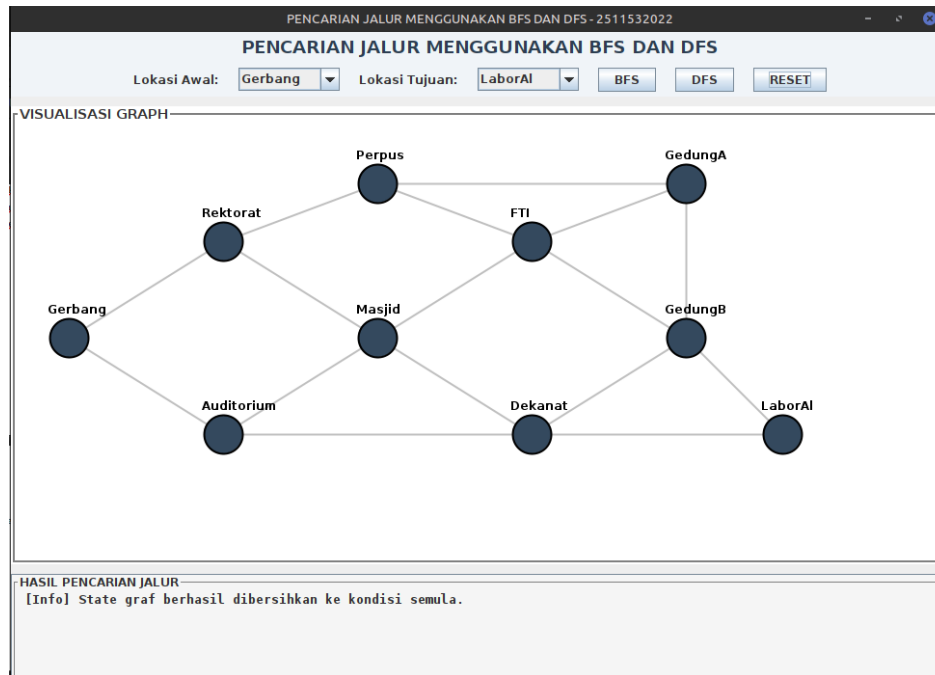
Penelusuran BFS:



Penelusuran DFS:



Reset Button ditekan



4. Penjelasan Struktur Graph, Cara Kerja BFS, dan DFS

4.1. Struktur Graph Peta Kampus Universitas Andalas

Studi kasus ini menggunakan Graf Tak Berarah (*Unweighted Undirected Graph*) untuk memetakan jalur transportasi di lingkungan Kampus Universitas Andalas. Graf ini direpresentasikan di dalam memori menggunakan struktur Senarai Ketetanggaan (*Adjacency List*).

- Komponen Utama Graf:
 - Simpul (*Vertex / V*): Berjumlah 10 node yang merepresentasikan lokasi fisik, yaitu: Gerbang, Rektorat, Auditorium, Masjid, Perpustakaan, FTI, Dekanat, GedungA, GedungB, dan Laboratorium.
 - Sisi (*Edge / E*): Berjumlah 16 garis penghubung dua arah yang menunjukkan adanya jalan akses langsung antar-lokasi.

- Mekanisme Penyimpanan Data:

Objek utama dideklarasikan sebagai `Map<String, List<String>>` `graph_2022`. Fungsi `HashMap` bertindak sebagai pencatat nama simpul asal (*Key*), sedangkan `ArrayList` menampung koleksi seluruh simpul tetangga terdekatnya (*Value*). Guna memastikan konsistensi urutan eksekusi, setiap daftar tetangga langsung diurutkan secara alfabetis menggunakan `Collections.sort()` saat inisialisasi graf dilakukan.

4.2. Cara Kerja Algoritma BFS (*Breadth-First Search*)

Algoritma BFS bekerja dengan prinsip penelusuran Melebar. Penelusuran dimulai dari simpul awal (*Start*) lalu mengunjungi seluruh simpul tetangga terdekat pada level yang sama terlebih dahulu, sebelum bergeser ke level kedalaman berikutnya.

- Struktur Data Pendukung: Menggunakan Antrean (*Queue*) berbasis prinsip *First-In, First-Out* (FIFO) dan Set untuk mencatat node yang sudah diperiksa (*Visited*).
- Langkah-Langkah Eksekusi (Studi Kasus: Gerbang > LaborAI):
 1. Node Gerbang dimasukkan ke dalam *Queue* dan ditandai sebagai *Visited*.
 2. Node Gerbang dikeluarkan (*poll*) dari antrean untuk dieksplorasi. Urutan kunjungan mencatat: [Gerbang].
 3. Tetangga dari Gerbang yang terurut alfabetis, yaitu Auditorium dan Rektorat, dimasukkan ke dalam *Queue* secara berurutan dan ditandai *Visited*.
 4. Node berikutnya di dalam antrean, yaitu Auditorium, dikeluarkan. Tetangganya (Dekanat dan Masjid) dimasukkan ke dalam *Queue*. Urutan kunjungan: [Gerbang -> Auditorium].

5. Node Rektorat dikeluarkan dari antrean. Tetangganya (Masjid dan Perpus) diperiksa. Karena Masjid sudah berstatus *Visited*, maka hanya Perpus yang dimasukkan ke antrean. Urutan kunjungan: [Gerbang -> Auditorium -> Rektorat].
6. Proses pelebaran level ini terus berulang hingga antrean menyentuh target node LaborAl. Begitu target tercapai, perulangan dihentikan (*break*).
7. Jalur solusi akhir dibentuk secara mundur menggunakan peta *Parent Map* untuk menghasilkan rute terpendek: Gerbang -> Auditorium -> Dekanat -> LaborAl.

4.3. Cara Kerja Algoritma DFS (*Depth-First Search*)

Algoritma DFS bekerja dengan prinsip penelusuran Mendalam. Penelusuran akan menyusuri satu cabang tetangga secara vertikal sedalam mungkin hingga mencapai titik buntu, sebelum melakukan proses lacak balik (*Backtracking*) untuk mencoba cabang tetangga lainnya.

- Struktur Data Pendukung: Menggunakan fungsi bantuan Rekursif yang secara implisit memanfaatkan memori *Call Stack* komputer dengan prinsip *Last-In, First-Out* (LIFO).
- Langkah-Langkah Eksekusi (Studi Kasus: Gerbang > LaborAl):
 1. Fungsi `dfsHelper_2022` dipanggil pertama kali dengan parameter node Gerbang. Node ditandai *Visited* dan masuk log kunjungan: [Gerbang].
 2. Algoritma langsung memeriksa tetangga pertama berdasarkan alfabet, yaitu Auditorium. Fungsi melakukan pemanggilan rekursif baru untuk Auditorium. Urutan kunjungan: [Gerbang -> Auditorium].

3. Dari Auditorium, tetangga belum dikunjungi berikutnya adalah Dekanat. Fungsi terjun lagi secara rekursif ke Dekanat. Urutan kunjungan: [Gerbang -> Auditorium -> Dekanat].
4. Dari Dekanat, tetangga alfabetis pertamanya adalah GedungB. Fungsi kembali melakukan penetrasi mendalam ke GedungB. Urutan kunjungan: [Gerbang -> Auditorium -> Dekanat -> GedungB].
5. Dari GedungB, tetangga pertamanya adalah FTI. Fungsi melompat ke FTI. Penelusuran berlanjut mendalam menembus node-node sisa secara beruntun (Masjid, Rektorat, Perpustakaan, GedungA) sebelum akhirnya tumpukan rekursif mengeksplorasi cabang menuju simpul target Laboratorium.
6. Karena sifat DFS pada program ini mengeksplorasi seluruh cabang *Call Stack* yang saling terhubung dalam satu kesatuan komponen graf, seluruh 10 Node akan tercatat ke dalam log urutan dikunjungi. Setelah selesai, jalur solusi ditarik mundur berdasarkan relasi *Parent* yang valid untuk membentuk rute akhir.